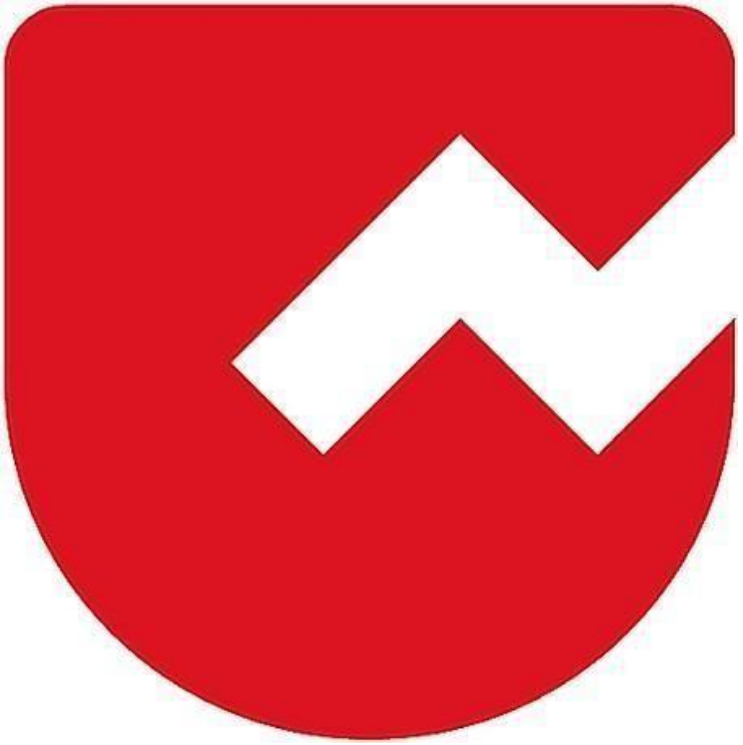
Zoho Schools for Graduate Studies



Notes

Date : 30/09/2025

Day : Tuesday

QUIZ - Methods

Question 1: Which of the following statement(s) describe the primary uses of methods in Java?

Options:
A) It allows code re-usability (define once and use multiple times).
B) You can break a complex program into smaller chunks of code.
C) It increases code readability and debugging easier.
D) All the above.

Correct Answer: D) All the above

Explanation:
Methods in Java improve **code reusability**, help **modularize complex programs**, and enhance **readability and debugging**, so all statements are correct.

Question 2: Which of the following statement is false?

Options:
A) Functions are defined independently of any object.
B) Methods are dependent and tied to an instance of a class (object) or the class itself.
C) Functions are called directly by name, whereas methods are called on an object or class.
D) None of the above.

Correct Answer: D) None of the above

Explanation:
All the given statements (A, B, C) are **true**, hence the false option is **"None of the above."**

Question 3: What is the output of the following code?

```
public class First {
    public static void main(String[] args) {
            add(3, 2);
    }
    public int add(int a, int b) {
```

```
        return a + b;
    }
}
```

Options:
A) 5
B) No Output
C) Compiler Error
D) Runtime Error

Correct Answer: C) Compiler Error

Explanation:
The add() method is **non-static**, but it is being called directly from the static main() method without creating an object. This leads to a **compile-time error**.

Question 4: What is the output of the following code?

```
public class First {
   public static void main(String[] args) {
      sayHello("Hello");
   }
   public static void sayHello(String s) {
      System.out.println(s);
   }
}
```

Options:
A) Hello
B) No Output
C) Compiler Error
D) Runtime Error

Correct Answer: A) Hello

Explanation:
The method sayHello() is **static**, so it can be called directly from the static main() method. It correctly prints "Hello" to the console.

Question 5: What is the output of the following code?

```
public class First {
    public static void main(String[] args) {
        new First().cube(3);
    }
    public static int cube(int n) {
        return n * n * n;
    }
}
```

Options:

A) 27
B) No Output
C) Compiler Error
D) Runtime Error

Correct Answer: B) No Output

Explanation:

The method cube(3) is executed, and the value **27** is computed, but it is **not printed** or stored. Since there is no System.out.println() statement, nothing is displayed on the console.

Question 6: What is the output of the following code?

```
public class First {
    public static void main(String[] args) {
        new First().fo();
        System.out.print("1");
    }
    public void fo() {
        System.out.print("2");
        go();
    }
    public void go() {
        ho();
        System.out.print("3");
    }
    public void ho() {
        System.out.print("4");
    }
}
```

Options:

A) 1234

B) 2431
C) 2341
D) 2413

Correct Answer: B) 2431

Explanation:
fo() prints 2, then calls go().go() calls ho(), which prints 4.After returning from ho(), go() prints 3. Finally, control goes back to main(), which prints 1.
So the output is 2431.

Question 7: Which of the following is true about the return statement in Java?

Options:
A) It can only return integers.
B) It can return multiple values.
C) It must always be the last statement in a method.
D) It can be used to return from a method at any point.

Correct Answer: D) It can be used to return from a method at any point.

Explanation:
In Java, the return statement is used to exit a method and optionally return a value. It can be placed anywhere in a method, not necessarily at the end. Methods can return only a single value (or an object/array containing multiple values), so options A, B, and C are incorrect.

Question 8: What is the output of the following code?

```
public class First {
   int x = 2;
  public static void main(String[] args) {
     int x = 3;
     int y = new First().go(x);
     System.out.print(x + y);
  }
   public int go(int y) {
     x = x + 2;
     return x + y;
  }
}
```

Options:
A) 5
B) 10
C) Compiler Error
D) Runtime Error
Correct Answer: B) 10

Explanation:
- First has an instance variable x = 2.
- In main, local x = 3 (different from instance x).
- go(x) is called with y = 3. Inside go, x = 2 + 2 = 4.
- The method returns x + y = 4 + 3 = 7.
- Back in main, x + y = 3 + 7 = 10.

Hence, the output is **10**.

Question 9: What is the output of the following code?

```
public class First {
   public static void main(String[] args) {
      if (args.length != 0) {
         System.out.println(args.length);
      } else {
         new First().go();
      }
   }
   public static void go() {
      ho();
   }
   public static void ho() {
      main(new String[] {"one", "two", "Three Four", "Five"});
   }
}
```

Options:
A) 0
B) 4
C) 5
D) Infinite loop

Correct Answer: B) 4

Explanation:

- The program starts with main and an empty args array.
- Since args.length == 0, it calls go().
- go() calls ho(), which calls main with a 4-element array: {"one", "two", "Three Four", " Five"}.
- Now args.length != 0, so it prints 4 and ends.
- No further recursion occurs.

Question 10: What is the output of the following code?

```
public class First {
   public static void main(String[] args) {
      System.out.println(go(go(go(3))));
   }
   public static int go(int n) {
      return 2 * n;
   }
}
```

Options:
A) 6
B) 12
C) 24
D) 48

Correct Answer: C) 24

Explanation:
- go(3) returns 2 * 3 = 6.
- go(go(3)) = go(6) = 2 * 6 = 12.
- go(go(go(3))) = go(12) = 2 * 12 = 24.
- Hence, the output is **24**.

Question 11: Which of the following statement(s) is False?

Options:
A) A parameter is a variable in the declaration of a function.
B) An argument is the actual value that is passed to the function when it is called.
C) Parameters act as placeholders for the actual values (arguments) that will be passed to the function when it is called.
D) The number of arguments provided when calling a function must match the number of parameters defined in the function declaration.

E) Parameters and arguments mean the same.

Correct Answer: E) Parameters and arguments mean the same.

Explanation:

- A **parameter** is a variable in a function declaration, while an **argument** is the actual value passed when calling the function.
- They are related but **not the same**, so the last statement is false.

QUIZ - Iterative Statements

Question 1: How many times will the following loop execute?

```
int n = 1;
do {
    n *= 2;
    } while (n < 100);
```

Options:

A) 6
B) 7
C) 8
D) 9

Correct Answer: B) 7

Explanation:

- n starts at 1 and doubles each time: 1 → 2 → 4 → 8 → 16 → 32 → 64 → 128.
- The loop stops when n < 100 is false.
- The loop executes **7 times** (last value 64, then doubles to 128 and exits).

Question 2: What is the output of the following code snippet upon execution?

```
int x = 10;
while (x-- > 5) {
```

```
    x -= 2;
    }
     System.out.println(x);
```

Options:

A) 2
B) 3
C) 4
D) 5

Correct Answer: B) 3

Explanation:

Step-by-step execution:

1. x = 10, condition x-- > 5 → 10 > 5 true, then **x becomes 9** after post-decrement, inside loop x -= 2 → x = 7.
2. x = 7, condition x-- > 5 → 7 > 5 true, then **x becomes 6**, inside loop x -= 2 → x = 4.
3. x = 4, condition x-- > 5 → 4 > 5 false, loop exits.

- The final value of x is **3** (because post-decrement reduces x by 1 after last condition check).

Question 3: What is the output of the following code snippet upon execution?

```
int count = 0;
for (int i = 0; i < 5; i++) {
    for (int j = i; j < 5; j++) {
        count++;
    }
}
System.out.println(count);
```

Options:

A) 20
B) 25
C) 15
D) None of the above

Correct Answer: C) 15

Explanation:

- Outer loop runs **i = 0 to 4** → 5 iterations.
- Inner loop runs from j = i to 4 → number of iterations decreases as i increases:

    - i=0 → 5 iterations
    - i=1 → 4 iterations
    - i=2 → 3 iterations
    - i=3 → 2 iterations
    - i=4 → 1 iteration
- Total increments of count = 5 + 4 + 3 + 2 + 1 = **15**.

Question 4: How many times will the following loop execute?_____

```
int count=0;
for (int i = 0; i < 10; i++) {
   for (int j = 10; j > i; j--) {
      // Some code
   }
}
```

Options:

A) 40
B) 55
C) 75
D) None of the above

Correct Answer: B) 55

Explanation:

- Outer loop: i = 0 to 9 → 10 iterations.
- Inner loop: j = 10 down to i+1 → decreases with each i.
- Number of inner loop executions:
    - i=0 → 10
    - i=1 → 9
    - i=2 → 8
    - …
    - i=9 → 1
- Total executions = 10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 = **55**

Question 5: What is the output of the following code snippet upon execution?

```
int sum = 0;
for (int i = 0; i < 3; i++) {
  for (int j = 0; j < 2; j++) {
     if (i == j) {
        continue;
```

```
        }
        sum += i + j;
    }
}
 System.out.println(sum);
```

Options:
A) 7
B) 10
C) 11
D) None of the above

Correct Answer: A) 7

Explanation:
Step-by-step execution:

- i=0:
  - j=0 → i==j → continue → skipped
  - j=1 → sum += 0+1 = 1 → sum = 1
- i=1:
  - j=0 → sum += 1+0 = 1 → sum = 2
  - j=1 → i==j → continue → skipped
- i=2:
  - j=0 → sum += 2+0 = 2 → sum = 4
  - j=1 → sum += 2+1 = 3 → sum = 7

Question 6: What is the output of the following code snippet upon execution?

```
int i = 0;
for (System.out.print("A"); i < 2; System.out.print("B"), i++)
        System.out.print("C");
```

Options:
A) Compiler Error
B) ABCBC
C) ACBCB
D) None of the above

Correct Answer: C) ACBCB

Explanation:
Step-by-step execution:

1. **Initialization:** System.out.print("A") → prints A
2. **Condition check:** i < 2 → 0 < 2 → true
3. **Body:** System.out.print("C") → prints C
4. **Update:** System.out.print("B"), i++ → prints B, then i = 1
5. **Condition check:** i < 2 → 1 < 2 → true
6. **Body:** System.out.print("C") → prints C
7. **Update:** System.out.print("B"), i++ → prints B, then i = 2
8. **Condition check:** i < 2 → false → loop ends

   Printed sequence: **ACBCB**

Question 7: What is the output of the following code snippet upon execution?

```
int sum = 0;
for (int i = 1; i <= 10; i++) {
   for (int j = 1; j <= i; j++) {
      sum += j;
   }
}
System.out.println(sum);
```

Options:

A) 100
B) 220
C) 330
D) None of the above

Correct Answer: B) 220

Explanation:

- Outer loop: i = 1 to 10
- Inner loop: j = 1 to i → sum of first i natural numbers is added in each iteration
- Total sum = 1 + 3 + 6 + 10 + 15 + 21 + 28 + 36 + 45 + 55 = **220**

Question 8: What is the output of the following code snippet upon execution?

```
int i = 0, j = 0, sum = 0;
for (; i < 5 && j < 3; i++, j++) {
      sum += i + j;
 }
 System.out.print(sum);
```

Options:
A) 6
B) 7
C) 8
D) None of the above

Correct Answer: A) 6

Explanation:
Loop trace:

- **Iteration 1:** i=0, j=0 → sum += 0+0 = 0
- **Iteration 2:** i=1, j=1 → sum += 1+1 = 2 (sum = 2)
- **Iteration 3:** i=2, j=2 → sum += 2+2 = 4 (sum = 6)
- Next, j=3, condition j < 3 fails → loop ends.

Question 9: What is the output of the following code snippet upon execution?

```
int result = 0;

for (int i = 1; i <= 50; i++) {
  if (i % 2 == 0) continue;
  result += i;
 }
 System.out.println(result);
```

Options:
A) 535
B) 625
C) 835
D) None of the above

Correct Answer: B) 625

Explanation:

- The continue skips even numbers.
- So only odd numbers from 1 to 49 are added.
- Sum of first n odd numbers = n2
- Here, number of odd terms = 25 (1, 3, 5, …, 49).
- Sum = 25^2 = 625.

Question 10: What is the output of the following code snippet upon execution?

```
int i = 0;
while (i < 3) {
    System.out.print(i);
    if (i == 2) break;
    i++;
}
```

Options:
A) 012
B) 0123
C) 0122
D) None of the above

Correct Answer: A) 012

Explanation:

- Start: i = 0 → prints 0, condition i == 2 false → i = 1.
- Next: i = 1 → prints 1, condition false → i = 2.
- Next: i = 2 → prints 2, condition true → break (loop ends).

Question 11: How many times will the innermost loop execute in the following code?

```
int count = 0;

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 2; j++) {
        for (int k = 0; k < 2; k++) {
            count++;
        }
    }
}
```

Options:
A) 12
B) 24
C) 10
D) None of the above

Correct Answer: B) 12

Explanation:

- Outer loop (i) → runs 3 times.
- Middle loop (j) → runs 2 times for each i.
- Inner loop (k) → runs 2 times for each (i, j) pair.
- Total executions = 3×2×2=12.

Question 12: What is the output of the following code snippet upon execution?

```
public class JumpStmt {
   public static void main(String []args) {
      int sum = 0;
      int j = 2;

      for (int i = 1; i <= 10; i++) {
         if (i % j == 0) {
            if (i % 5 == 0)
               break;
            sum += i;
            System.out.print(i + " ");
         } else {
            continue;
         }
      }

      System.out.println("Sum=" + sum);
   }
}
```

Options:

A) 30
B) 20
C) 10
D) 0

Correct Answer: B) 20

Explanation:

- j = 2, so we only consider even numbers.
- Loop values:
  - i=1 → odd → skipped
  - i=2 → even, not multiple of 5 → sum = 2
  - i=3 → odd → skipped
  - i=4 → even, not multiple of 5 → sum = 2+4=6
  - i=5 → odd → skipped

    - i=6 → even, not multiple of 5 → sum = 6+6=12
    - i=7 → odd → skipped
    - i=8 → even, not multiple of 5 → sum = 12+8=20
    - i=9 → odd → skipped
    - i=10 → even, divisible by 5 → **breaks loop**
- Final sum = **20**

Question 13: What is the output of the following code snippet upon execution?

```
public class JumpStmt {
   public static void main(String []args) {
      int i = 10;

      outer:
      while (true) {
         if (i == 0) {
            i--;
            break outer;
         }
         System.out.print(i + " ");
      }
   }
}
```

Options:

A) Runtime Error
B) Compiler Error
C) Infinite Loop
D) None of the above

Correct Answer: C) Infinite Loop

Explanation:

- i is initialized as 10.
- Inside the while(true) loop:
    - Condition if (i == 0) is never true because i is never decremented.
    - Hence, the break outer; is **never reached**.
- Loop keeps printing 10 10 10 ... forever.

Question 14: What is the output of the following code snippet upon execution?

```
      int sum = 0;
      for (int i = 0; i < 5; i++) {
```

```
            for (int j = 0; j < 5; j++) {
                if (i == j)
                    break;
                else
                    sum += i + j;
            }
        }

        System.out.println(sum);
    }
}
```

Options:

A) 25
B) 40
C) Infinite Loop
D) None of the above

Correct Answer: B) 40

Explanation:

Let's trace the loops:

- **i = 0** → inner loop: j=0, condition i==j → break immediately → no addition.
- **i = 1** → inner loop: j=0 → sum += 1+0 = 1 → next j=1 → break → sum = 1.
- **i = 2** → inner loop: j=0 → sum=1+(2+0)=3, j=1 → sum=3+(2+1)=6, j=2 → break → sum=6.
- **i = 3** → inner loop: j=0 → sum=6+3=9, j=1 → sum=9+4=13, j=2 → sum=13+5=18, j=3 → break → sum=18.
- **i = 4** → inner loop: j=0 → sum=18+4=22, j=1 → sum=22+5=27, j=2 → sum=27+6=33, j=3 → sum=33+7=40, j=4 → break → sum=40.

Question 15: What is the output of the following code snippet upon execution?

```
public class JumpStmt {
    public static void main(String[] args) {
        for (int i = 1; i < 5; i++) {
            if (i % (i - 1) == 1)
                break;
            System.out.println(i);
        }
    }
}
```

Options:
A) Compiler Error
B) Runtime Error
C) Infinite Loop
D) None of the above

Correct Answer: B) Runtime Error

Explanation:
- When i = 1, the condition i % (i - 1) becomes 1 % 0, which **divides by zero**.
- In Java, dividing by zero during runtime throws an **ArithmeticException**.
- Hence, the program terminates with a **Runtime Error**.